FIAP

GRADUAÇÃO

45697056

ES
Desenvolvimento Mobile

Prof. Adeilton Meneses

profadeilton.meneses@fiap.com.br

Introdução aos Hooks no React Native

45697056

- **O que são Hooks?**
  - Hooks são funções especiais no React que permitem que você "engate" no estado e no ciclo de vida de um componente de função. Antes dos Hooks, essas funcionalidades estavam disponíveis apenas em componentes de classe.

Documentação: https://react.dev/reference/react/hooks

Introdução aos Hooks no React Native

45697056

- **O que são Hooks?**
  - Com a introdução dos Hooks na versão 16.8 do React, tornou-se possível escrever componentes totalmente funcionais que gerenciam seu próprio estado e efeitos colaterais (como requisições a APIs), tornando o código mais conciso e reutilizável.
  - Resolver a complexidade dos componentes de classe, o compartilhamento de lógica com `render props` e `higher-order components`.

Documentação: https://react.dev/reference/react/hooks

Introdução aos Hooks no React Native

45697056

- **Por que usar Hooks?**

  Os Hooks trazem várias vantagens:

  - **Reutilização de lógica com estado**: Com Hooks personalizados (*custom hooks*), você pode extrair a lógica de estado de um componente e reutilizá-la em vários outros.
  - **Código mais limpo**: Eles ajudam a organizar a lógica relacionada ao estado em um só lugar, em vez de espalhá-la por diferentes métodos de ciclo de vida, como acontece em componentes de classe.
  - **Componentes de função**: Permitem que você use o React sem a necessidade de componentes de classe, que são mais complexos de entender e usar.
  - **Simplificação do código**: Elimina a necessidade de classes e do uso de `this`.

Hooks Essenciais

45697056

- *useState*
- *useEffect*
- *useContext*
- *Criar seu Hook (Custom Hook)*

Hooks Essenciais - *useState*

45697056

- **Conceito:** O Hook mais fundamental. Ele adiciona o estado a um componente funcional.

- **Sintaxe:** **const [state, setState] = useState(initialState);**
  - **state**: A variável de estado.
  - **setState**: A função para atualizar o estado.
  - **initialState**: O valor inicial do estado.

Exemplos Práticos

45697056

Gerenciador de Tarefas (useState)

- **Objetivo:** Construir uma lista de tarefas.

- **Requisitos:**
  - Um **TextInput** para adicionar novas tarefas.
  - Um botão para adicionar a tarefa à lista.
  - Exibir a lista de tarefas em um **FlatList**.
  - Usar **useState** para gerenciar a lista de tarefas e o texto de entrada.

Validação de Formulários com `useState`

Usando o Hook *`useState`* para Controle de Estado e Erros

INTRODUÇÃO

Por que a Validação é Crucial?

- **Qualidade dos Dados:** Garante que apenas dados corretos e completos cheguem ao backend.
- **Experiência do Usuário (UX):** Fornece feedback imediato, evitando frustrações.
- **Segurança:** Evita a submissão de dados maliciosos ou formatação incorreta.

- **O Desafio no React Native:** Gerenciar múltiplos estados (value e error) de forma síncrona.

FUNDAMENTOS: ESTRUTURA DO ESTADO

- A Dupla do `useState`: Valor + Erro
- **Conceito-Chave:** Cada campo de formulário precisa de duas variáveis de estado.

Código de Exemplo:

```
// 1. Estado para o valor do campo
const [email, setEmail] = useState('');

// 2. Estado para a mensagem de erro
const [emailError, setEmailError] = useState(null); // null é 'sem erro'
```

- **Implementação no Input:**
  - O `value` é alimentado pelo estado (`email`).
  - O `onChangeText` atualiza o estado (`setEmail`).

VALIDAÇÃO EM TEMPO REAL (`onChangeText`)

- Feedback Imediato: Validando a Cada Digitação
- Guiar o usuário enquanto ele preenche o formulário.
- **Processo:**
  1. O usuário digita no `TextInput`.
  2. O `onChangeText` é acionado.
  3. Chama a função de validação **imediatamente**.
  4. Atualiza o estado de erro (`setEmailError`).

Exemplo de Lógica:

```
const handleEmailChange = (text) => {
  setEmail(text);
  if (text.length < 5) {
    setEmailError('Mínimo de 5 caracteres.');
  } else {
    setEmailError(null); // Limpa o erro
  }
};
```

VALIDAÇÃO FINAL (SUBMISSÃO)

- `handleSubmission`: A Verificação Crítica
- **Gatilho:** O `onPress` do botão de envio.
- **Função:** Deve revalidar **TODOS** os campos uma última vez, garantindo que não há campos vazios ou pendentes.

Lógica Centralizada:

```
const handleSubmit = () => {
  // 1. Revalidações forçadas...
  // 2. Verifica se algum erro existe:
  if (emailError || passwordError || !email) {
    alert('Corrija os erros antes de enviar.');
    return;
  }

  // 3. Se tudo OK:
  console.log('Dados prontos para o API!');
};
```

FEEDBACK VISUAL

- Melhores Práticas de UX (Experiência do Usuário)
- **1. Estilização Condicional do Input:**
  - Use o estado de erro para mudar a cor da **borda** do `TextInput` para vermelho (por exemplo).
  - **Dica:** `style={[styles.input, emailError && styles.inputError]}`
- **2. Exibição da Mensagem de Erro:**
  - Use **Renderização Condicional** para mostrar o texto do erro.

Exemplo:

```
{emailError && <Text style={styles.errorText}>{emailError}</Text>}
```

3. Confirmação de Sucesso (Toast):

- Utilize o `react-native-toast-message` após uma submissão bem-sucedida para notificar o usuário;
- Utilize um alert;
- Personalize um modal;

Bibliotecas para validação de formulário

O gerenciamento de estado e validação de formulários grandes exige ferramentas que reduzem o código repetitivo.

Categoria	Biblioteca	Foco Principal	Melhor Combinação
Gerenciador (Controla Estado e Submissão)	**React Hook Form (RHF)**	**Performance** (Mínimas re-renders).	**RHF + Zod** (Performance e TypeScript)
Gerenciador	**Formik**	**Abstração total** e facilidade para iniciantes.	**Formik + Yup** (Solução completa e madura)
Validador (Define as Regras)	**Zod**	**Type-Safety** (TypeScript) e clareza.	Compartilhável entre Front/Backend.
Validador	**Yup**	Sintaxe fluente e mais usada no ecossistema React.	Simplifica a definição de regras complexas.

- **Quando usar o `useState` Puro?**

  Apenas para formulários pequenos **5 campos** e **lógica de validação muito simples**. Qualquer coisa mais complexa deve usar as bibliotecas acima.

Bibliotecas para validação de formulário

Aspecto	Manual	Formik + Yup	React Hook Form + Zod
Complexidade	Baixa	Média	Média
Performance	Boa	Boa	Excelente
Bundle Size	Menor	Maior	Médio
TypeScript	Manual	Boa	Excelente
Validação	Manual	Declarativa	Declarativa
Re-renders	Muitos	Muitos	Poucos
Curva de Aprendizado	Baixa	Média	Média

Formulário Manual

- Projetos simples com poucos campos
- Quando você precisa de controle total
- Para aprender os conceitos básicos
- Não é ideal para Formulários complexos com muitas validações

Formik + Yup

- Projetos que já usam Formik
- Validações complexas e condicionais
- Equipes familiarizadas com Yup
- Quando precisa de muitas validações customizadas

React Hook Form + Zod

- Projetos novos com TypeScript
- Performance é crítica
- Validação com tipos seguros
- Formulários com muitos campos
- Aplicações que precisam de menos re-renders

Bibliotecas para validação de formulário

- ***React Hook Form*** (Gestão de Formulários): https://www.react-hook-form.com/api/

- ***Zod*** (Validação de Esquemas): https://zod.dev/basics

- ***Formik*** (Gestão de Formulários): https://formik.org/docs/api/formik

- ***Yup*** (Validação de Esquemas): https://github.com/jquense/yup (a biblioteca Yup é um construtor de esquemas e seu repositório GitHub é a fonte principal de referência.)

Vamos conhecer o código?

Criando projeto

45697056

- Criando nosso projeto React native + Expo
  - npx create-expo-app --template
  - selecionar template Blank (Typescript)
  - definir nome do projeto
  - entrar na pasta criada (cd "nome da pasta")
  - npm start
  - escolher plataforma
    - **A** - android
    - **I** - iOS

Explicação comandos

45697056

npx create-expo-app - -template

- Criar um novo projeto Expo usando o template blank ou de sua preferência.
- npx: executa pacotes sem precisar instalá-los globalmente.
- create-expo-app: ferramenta para iniciar projetos Expo.
- –template: lista templates (com React Native + Expo).

45697056

Projeto base

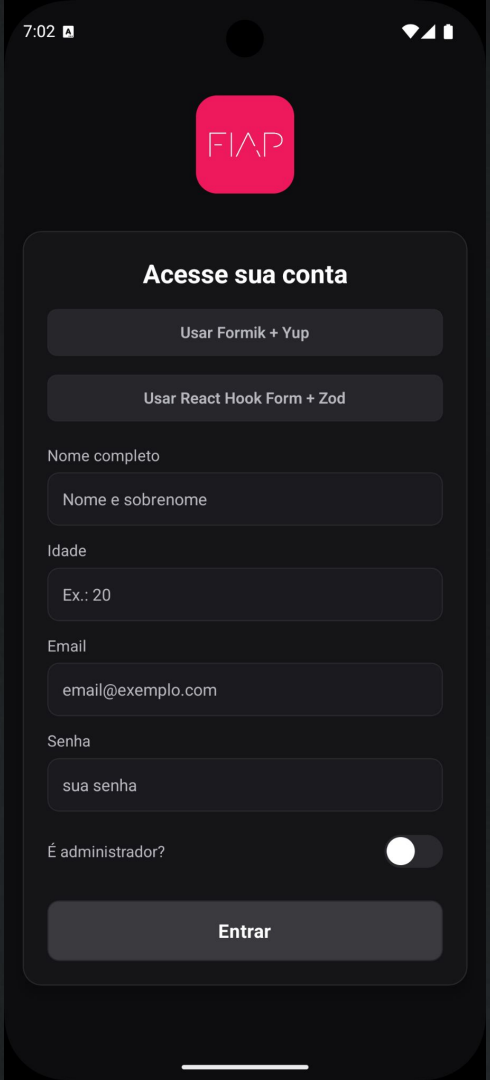
https://github.com/admeneses/form-initial

Como executar a primeira vez:

- clonar o projeto
- npm i

Depois de configurado:

npm start + (a plataforma que será executado)



Obrigado

2025-Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proíbido sem o consentimento formal, por escrito, do Professor Wellington Cidade Silva Tenorio.